SIMATS ENGINEERING

ASSESSMENT TOOL 1: Analytical Problem Solving

COURSE NAME: Cloud Computing and Big Data Analytics

COURSE CODE: CSA1513

COURSE OUTCOME COVERED

CO2: *Analyze virtualization architectures to identify performance and security issues in cloud computing environments.* (BL3)

Dave's Taxonomy: Manipulation (Level 2)

Total Marks: 50

Question Count: 5

Duration : 60 Minutes

Code of Conduct

I K. Ragavi (192524364) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result indisciplinary action.

Signature of the student: K. Ragavi

Assessment Task

Analytical Problems Solving

Question 1

Compare Type-1 and Type-2 hypervisors for a cloud datacenter hosting mission-critical applications.

- Analyze performance, security, and manageability, then justify the better choice.

Answer

1. Introduction

A hypervisor is the software layer that creates and runs virtual machines by sharing physical CPU, memory, storage and network among them. A Type-1 (bare-metal) hypervisor, such as VMware ESXi, Microsoft Hyper-V, Xen or KVM, runs directly on the server hardware. A Type-2 (hosted) hypervisor, such as VMware Workstation or VirtualBox, runs as an application on top of a general-purpose host operating system.

2. Comparison

Parameter	Type-1 (Bare-metal)	Type-2 (Hosted)
Position in stack	Hardware → Hypervisor → VMs	Hardware → Host OS → Hypervisor → VMs
Performance	Direct hardware access, low overhead, near-native speed; supports hardware assist (VT-x/AMD-V, SR-IOV)	Extra OS layer adds overhead and latency; resources contend with host OS processes
Security	Small attack surface (minimal code base); isolation enforced in hardware; no general-purpose apps on host	Larger attack surface: host OS vulnerabilities, malware or other apps can compromise all VMs
Manageability	Centralized management (vCenter, SCVMM), live migration, HA, DRS, automation and APIs for large clusters	Managed manually per machine; no clustering, limited automation
Scalability and reliability	Hundreds of VMs per host, fault tolerance, failover	Limited VM count; host OS crash or reboot stops all VMs
Typical use	Datacenters and public/private clouds	Desktops, testing, learning, development

3. Analysis

- **Performance:** Mission-critical applications need predictable low latency and high throughput. Type-1 avoids the host OS scheduling and I/O path, so CPU and I/O performance is close to bare metal.
- **Security:** Fewer layers mean fewer vulnerabilities. In Type-2, one exploit of the host OS can expose every VM, which is unacceptable for critical data.
- **Manageability:** Clustering, live migration, automated failover and centralized patching reduce downtime and operational effort in a datacenter.

4. Justification and Conclusion

A Type-1 hypervisor is the better choice for a cloud datacenter hosting mission-critical applications because it provides higher performance, stronger isolation, better resource utilization and enterprise-class availability features. Type-2 hypervisors are suitable only for development, testing and training. Hence, the datacenter should deploy a Type-1 hypervisor such as ESXi, Hyper-V or KVM.

Screenshot suggestion: Library pane showing multiple VMs. Caption: "Multiple VMs running on one hypervisor host, representing multiple cloud instances."

Question 2

A cloud provider must isolate workloads belonging to finance, healthcare, and student portals on the same hardware.

- Analyze how virtualization architecture supports isolation and identify two major attack surfaces.

Answer

1. Problem Interpretation

Three tenants with different sensitivity and compliance needs (finance, healthcare, student data) share the same physical servers. The provider must make sure that one tenant cannot read, modify or disrupt another tenant's data or performance (multi-tenancy isolation).

2. How Virtualization Architecture Supports Isolation

- **Hypervisor isolation:** Each workload runs in its own VM with its own guest OS, virtual CPU, memory and disk. The hypervisor, using hardware features (Intel VT-x/AMD-V, EPT/NPT, IOMMU), prevents a VM from accessing another VM's memory.
- **Resource isolation:** Reservations, limits and shares of CPU, memory and I/O prevent a noisy neighbour from starving other tenants.
- **Network isolation:** Separate virtual switches, VLAN/VXLAN segments, virtual firewalls and micro-segmentation keep traffic of each portal apart.
- **Storage isolation:** Separate virtual disks or volumes, per-tenant encryption keys and LUN masking.
- **Management isolation:** Role-based access control, separate management network, and separate resource pools/projects per tenant.
- Dedicated hosts or host groups (anti-affinity rules) can be used for the highly regulated healthcare and finance workloads.

3. Two Major Attack Surfaces

- **Hypervisor / VM escape:** A vulnerability in the hypervisor, virtual devices or drivers could let an attacker in one VM break out and control the host or access VMs of other tenants (for example, a compromised student-portal VM attacking the finance VM). Related risks include side-channel attacks (cache timing, Spectre-type) that leak data across VMs sharing a CPU.
- **Management plane and virtual network:** The hypervisor management interfaces (consoles, APIs, vCenter), stolen admin credentials, and the virtual network (VM-to-VM traffic, misconfigured switches/VLANs, VM snapshots and images) can give attackers broad access to many tenants at once.

4. Countermeasures

- Patch hypervisors regularly, disable unused virtual devices, and use hardened minimal hypervisors with secure boot.
- Use MFA and RBAC for administrators, and a separate, firewalled management network.
- Enforce micro-segmentation, encrypt VM disks, snapshots and traffic, and monitor logs for anomalies.

- Use dedicated hosts or hardware-level isolation (confidential computing) for the most sensitive tenants.

Conclusion: Virtualization provides strong logical isolation, but security depends on a well-patched hypervisor, secured management interfaces and properly segmented virtual networks.

Screenshot suggestion: Library pane with three VMs (Finance, Healthcare, Student) on different virtual networks. Caption: "Separate VMs with isolated virtual networks on the same host."

Question 3

Study the following VM host utilization snapshot and recommend corrective actions: CPU 92%, memory 88%, storage I/O latency 35 ms, average VM boot time 170 s.

- Explain the likely bottlenecks.

Answer

1. Interpretation of the Snapshot

Metric	Value	Typical healthy level	Assessment
CPU utilization	92%	Below 70–75%	Critically high; possible CPU contention, with VMs waiting for CPU time (high CPU ready time)
Memory utilization	88%	Below 80–85%	High; risk of ballooning, swapping and compression, which slows VMs
Storage I/O latency	35 ms	Below 10–20 ms	Poor; storage is a bottleneck (SSD should be under a few ms)
Average VM boot time	170 s	Around 30–60 s	Very slow; caused by combined resource contention

2. Likely Bottlenecks and Cause-Effect Analysis

- **CPU over-commitment:** Too many vCPUs are allocated per physical core, so VMs queue for CPU; this slows applications and boot.
- **Memory pressure:** At 88%, the hypervisor may use ballooning or swap to disk, which in turn adds more storage I/O and further slows performance.
- **Storage I/O:** High latency suggests slow disks (HDD), saturated storage controller or network, or many VMs doing I/O at once. This is the largest contributor to the long boot time, since boot is mainly disk reads.
- **Boot storm:** Many VMs booting simultaneously create heavy random I/O, magnifying the above problems.
- **Overall:** CPU, memory and storage contention are interacting: memory shortage causes swapping, swapping raises I/O latency, and high latency keeps vCPUs waiting, making the host overloaded.

3. Corrective Actions

- **Rebalance workloads:** Use live migration (vMotion/DRS) to move VMs to less-loaded hosts and add hosts to the cluster for capacity.
- **Right-size VMs:** Reduce excess vCPUs and RAM of oversized VMs; set reasonable limits and reservations for critical VMs, and lower the vCPU-to-core ratio.

- **Memory:** Add RAM, enable memory sharing/compression, remove idle VMs, and avoid overcommitting beyond safe limits.
- **Storage:** Move VMs to SSD/NVMe tiers, use caching or tiered storage, distribute VMs across datastores, enable multipathing, and use thin clones or linked clones for faster boot.
- Stagger boot-up schedules to avoid boot storms; use templates and optimized images.
- Enable monitoring and alerts (CPU ready, ballooning, latency) and use auto-scaling and capacity planning to prevent recurrence.

Conclusion: The host is overloaded in CPU, memory and storage; the storage latency is the primary reason for the slow boot. Rebalancing, right-sizing and faster storage should bring utilization below 75% and latency under 10–20 ms.

Screenshot suggestion: VM → Manage → Clone, or Snapshot Manager. Caption: "Cloning a VM to spread load and Snapshot Manager showing quick recovery."

Question 4

Analyze how Software Defined Datacenter architecture improves agility compared with a traditional datacenter.

- Support your answer with compute, storage, and network virtualization considerations.

Answer

1. Introduction

A Software Defined Datacenter (SDDC) virtualizes all infrastructure (compute, storage, networking and security) and manages it through software, with automation and policy-driven orchestration over standard hardware. In a traditional datacenter, hardware is tightly bound to applications and configured manually, device by device. Agility means the ability to provision, change and scale resources quickly.

2. Comparison

Aspect	Traditional datacenter	Software Defined Datacenter
Provisioning	Days or weeks; manual procurement and setup	Minutes; automated through templates and APIs
Hardware	Proprietary, siloed	Commodity hardware abstracted by software
Management	Device-by-device, CLI based	Centralized, policy-based, programmable
Scaling	Over-provisioning, slow scale-up	Elastic scale-out on demand
Utilization	Low (often 15–30%)	High through pooling and sharing
Cost	High CapEx and administration	Lower CapEx/OpEx through automation

3. Virtualization Considerations

- **Compute virtualization:** Hypervisors pool CPU and memory and create VMs or containers in seconds. Live migration, HA and DRS balance load and avoid downtime, while templates and cloning allow fast deployment and auto-scaling.
- Storage virtualization (software-defined storage): Local disks across servers form a shared pool (e.g., vSAN, Ceph). Policies define performance, redundancy and capacity per VM; thin provisioning, snapshots and tiering allow instant volume creation and growth without storage-team intervention.
- **Network virtualization (SDN/NFV):** Virtual switches, routers, load balancers and firewalls are created in software with overlays such as VXLAN. The control plane is separated from the data plane and centralized, so whole networks and security policies (micro-segmentation) are created or changed by script instead of rewiring.

4. How Agility Improves

- Self-service portals and Infrastructure as Code deliver complete environments quickly.

- Resources scale up or down automatically with demand, improving time-to-market.
- Policy-driven automation reduces human error and enforces consistent security.
- Easier hybrid-cloud integration, workload mobility, and faster disaster recovery through replication of virtual infrastructure.

Considerations: SDDC needs skilled staff, strong management-plane security and careful capacity planning, but it offers far greater agility than a traditional datacenter.

Screenshot suggestion: Edit → Virtual Network Editor. Caption: "Virtual networks (NAT, Host-only, Bridged) created in software, demonstrating network virtualization."

Question 5

A multi-cloud federation project fails due to identity mismatches between providers.

- Analyze the issue and propose a secure identity and privacy design using federation concepts.

Answer

1. Analysis of the Issue

Each cloud provider has its own identity system (AWS IAM, Azure AD/Entra ID, Google Cloud IAM) with different user formats, roles, attribute names, token types and authentication methods. When federating them, the following mismatches commonly cause failure:

- Different identifiers and naming (e.g., email vs UPN vs username), causing the same user to be treated as different identities or duplicate accounts.
- Inconsistent roles and permission models, so authorization rules do not translate between providers.
- Incompatible protocols or token formats (SAML vs OIDC/OAuth2) and unsynchronized attributes or claims.
- Separate user directories with no single source of truth; out-of-date or orphaned accounts after staff changes.
- **Trust-configuration problems:** mismatched certificates, clock skew, wrong audience or endpoints.
- Over-privileged access and exposure of personal data because attributes are shared without control.

2. Proposed Secure Identity and Privacy Design

- **Central Identity Provider (IdP):** Use one authoritative IdP (e.g., Azure AD, Okta or an enterprise directory) as the source of truth for all identities.
- **Federation and SSO:** Each cloud acts as a Service Provider that trusts the IdP using SAML 2.0 or OpenID Connect/OAuth 2.0. Users authenticate once and receive signed tokens or assertions; no passwords are shared with the clouds.
- **Identity mapping and standardization:** Define a common attribute schema and claims (unique ID, group, role), and map federated groups to provider-specific roles (for example, group "Finance-Admin" to an AWS role and an Azure role).
- **Provisioning:** Use SCIM for automatic user creation, update and deprovisioning, preventing stale accounts.
- **Strong authentication and authorization:** MFA, RBAC/ABAC, least privilege, conditional access and short-lived tokens; just-in-time privileged access.
- **Privacy controls:** Share only the minimum required attributes (data minimization), use pseudonymous or pairwise identifiers, user consent, and comply with GDPR and similar rules.
- **Security of the trust:** Sign and encrypt assertions, rotate certificates and keys, synchronize time (NTP), and use TLS.

- **Governance:** Central audit logs, periodic access reviews and monitoring of federated sign-ins.

3. Expected Outcome

With a single trusted IdP, consistent claims and automated provisioning, users get seamless and secure access across all providers, mismatches are removed, access can be revoked in one place, and personal data exposure is reduced. The federation project then becomes reliable, auditable and compliant.

Screenshot suggestion: Virtual Network Editor with NAT/Host-only/Bridged networks and multiple VMs. Caption: "Connectivity between separate environments, representing hybrid or multi-cloud federation."

Analytical Problem Solving Rubric

Criteria	Weightage	Level 4 - Exemplary	Level 3 – Proficient	Level 2 - Developing	Level 1 - Beginning
Problem Interpretation	20%	Interprets all technical requirements accurately	Interprets most requirements correctly	Partial interpretation	Misinterprets the problem
Analytical Reasoning	25%	Strong reasoning with clear cause-effect analysis	Good reasoning with minor gaps	Basic analysis only	Weak or no analysis
Method Selection	20%	Chooses highly appropriate virtualization and security concepts	Chooses mostly suitable concepts	Partially suitable concepts	Inappropriate approach
Solution Quality	20%	Provides feasible and technically sound solution	Reasonable solution with minor issues	Basic solution with limitations	Unclear or invalid solution
Presentation of Analysis	15%	Well-structured and technically precise	Generally clear	Somewhat unclear	Difficult to follow